

SML Assignment 2

Emmanuel Ansah

12/14/2024

Packages Used

```
## corrrplot 0.95 loaded
## Loading required package: lattice
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
## The following objects are masked from 'package:stats':
##
##      cov, smooth, var
```

Exercise 1: Practical SML on DNA Microarrays

```
prostate <- read.csv("prostate-cancer-1.csv")
```

1. Comment on the shape of this dataset in terms of the sample size and the dimensionality of the input space

```
dim(prostate)
```

```
## [1] 79 501
```

The dataset has 79 rows and 501 columns. The dimensionality of the input space is very high at 500 variables, while the sample size is small at 79. This means that there are far more features than samples in the dataset, making it a high-dimensional dataset.

2. Comment succinctly from the statistical perspective on the type of data in the input space.

```
str(prostate[, 1:10])
```

```
## 'data.frame': 79 obs. of 10 variables:
## $ Y : int 0 0 0 0 0 0 0 0 0 0 ...
## $ X206212_at : num -0.2177 -0.138 -0.1475 -0.1907 -0.0992 ...
## $ X207075_at : num -0.34 -0.266 -0.262 -0.326 -0.208 ...
## $ X215872_at : num -0.354 -0.277 -0.262 -0.326 -0.247 ...
## $ X201876_at : num 0.302651 0.09088 0.028023 0.393919 0.000499 ...
## $ X211935_at : num 0.615 0.113 0.352 0.412 0.416 ...
```

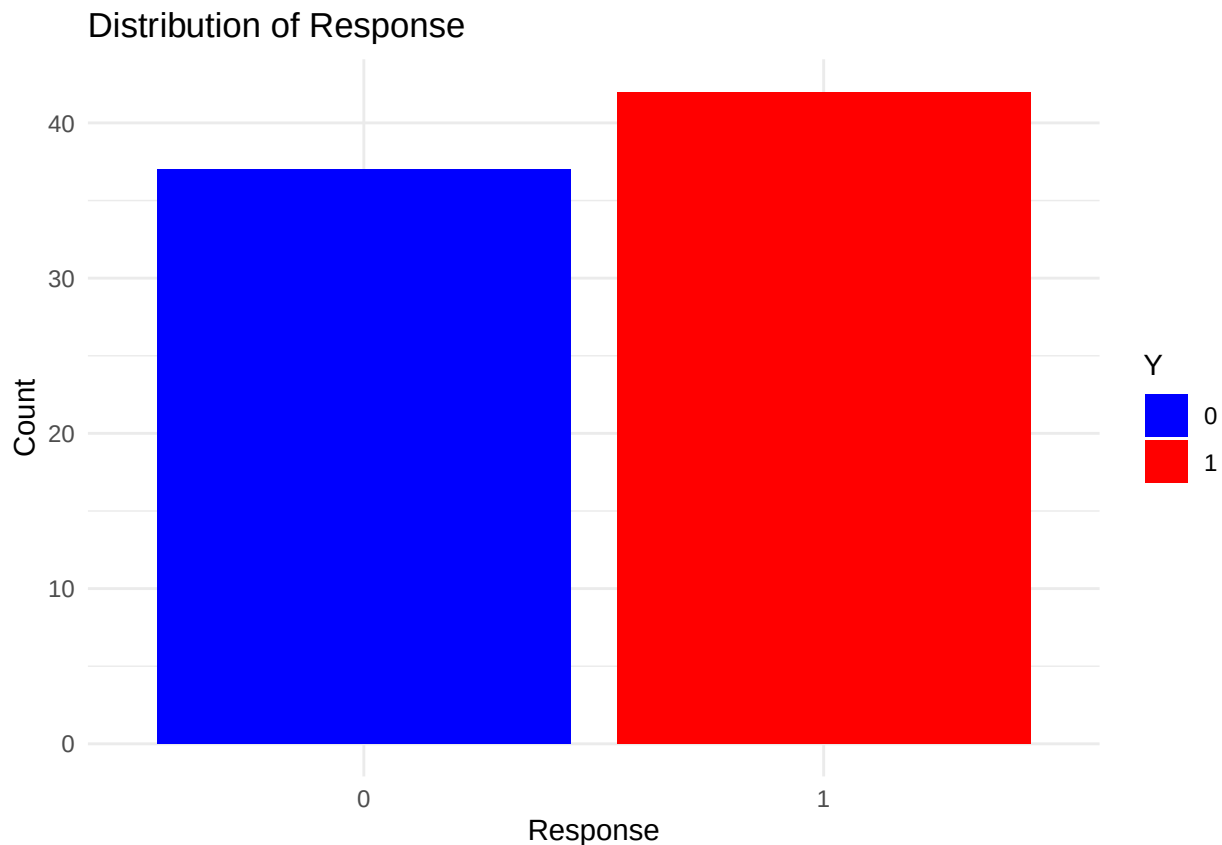
```
## $ X206788_s_at: num -0.154 -0.175 -0.195 -0.18 -0.165 ...
## $ X216441_at : num -0.356 -0.283 -0.261 -0.326 -0.242 ...
## $ X209290_s_at: num 0.3463 0.2859 0.2852 0.295 0.0996 ...
## $ X219877_at : num -0.348 -0.28 -0.255 -0.324 -0.239 ...
```

The input data are gene expression levels measured from DNA microarrays, making it continuous numeric data. Each variable represents the expression level of a particular gene.

3. Plot the distribution of the response for this dataset and comment.

```
prostate$Y <- as.factor(prostate$Y)

ggplot(prostate, aes(x = Y, fill = Y)) +
  geom_bar() +
  labs(title = "Distribution of Response", x = "Response", y = "Count") +
  scale_fill_manual(values = c("blue", "red")) +
  theme_minimal()
```



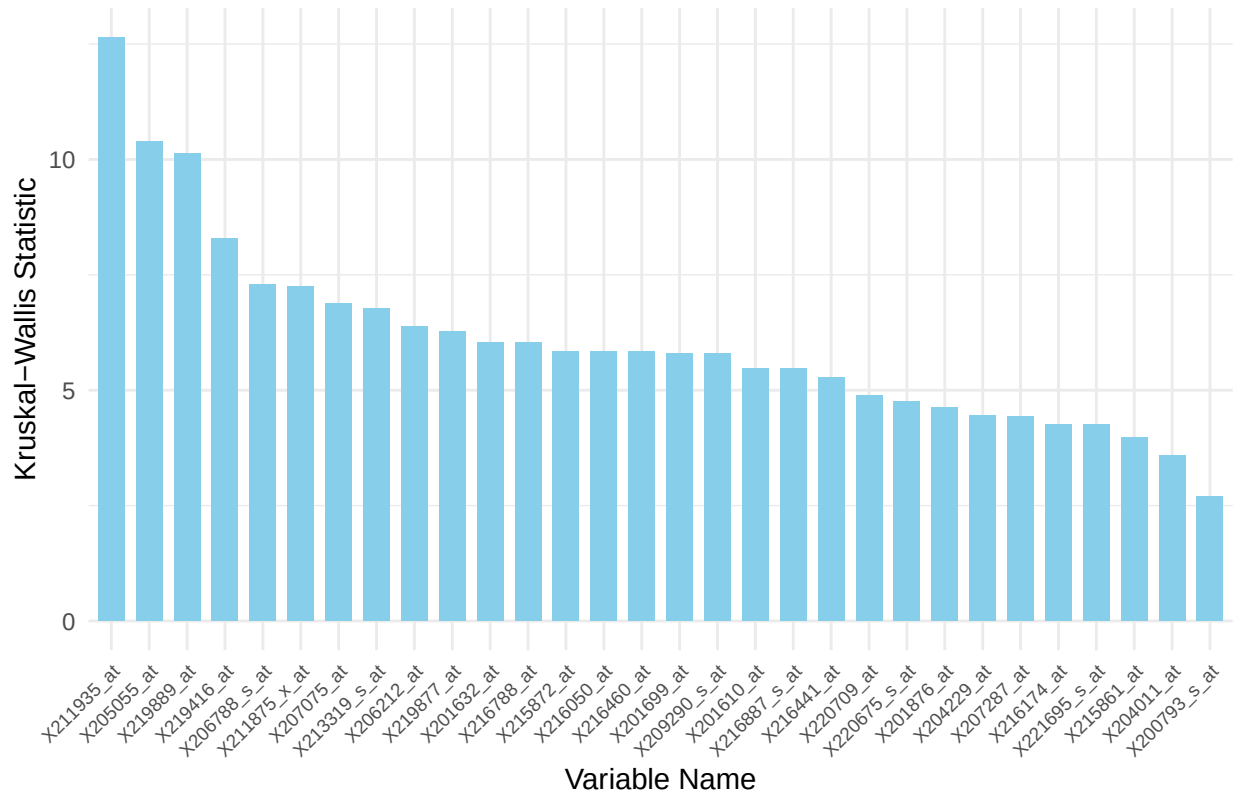
The plot for distribution of the response shows that the response variable (Y) which represents whether the subjects have cancer or not, is relatively balanced, meaning there are approximately similar numbers of subjects in both the cancer and non-cancer groups. We do not have to worry about class imbalance.

4. Identify the 9 individually most powerful predictor variables with respect to the response according the Kruskal-Wallis test statistic

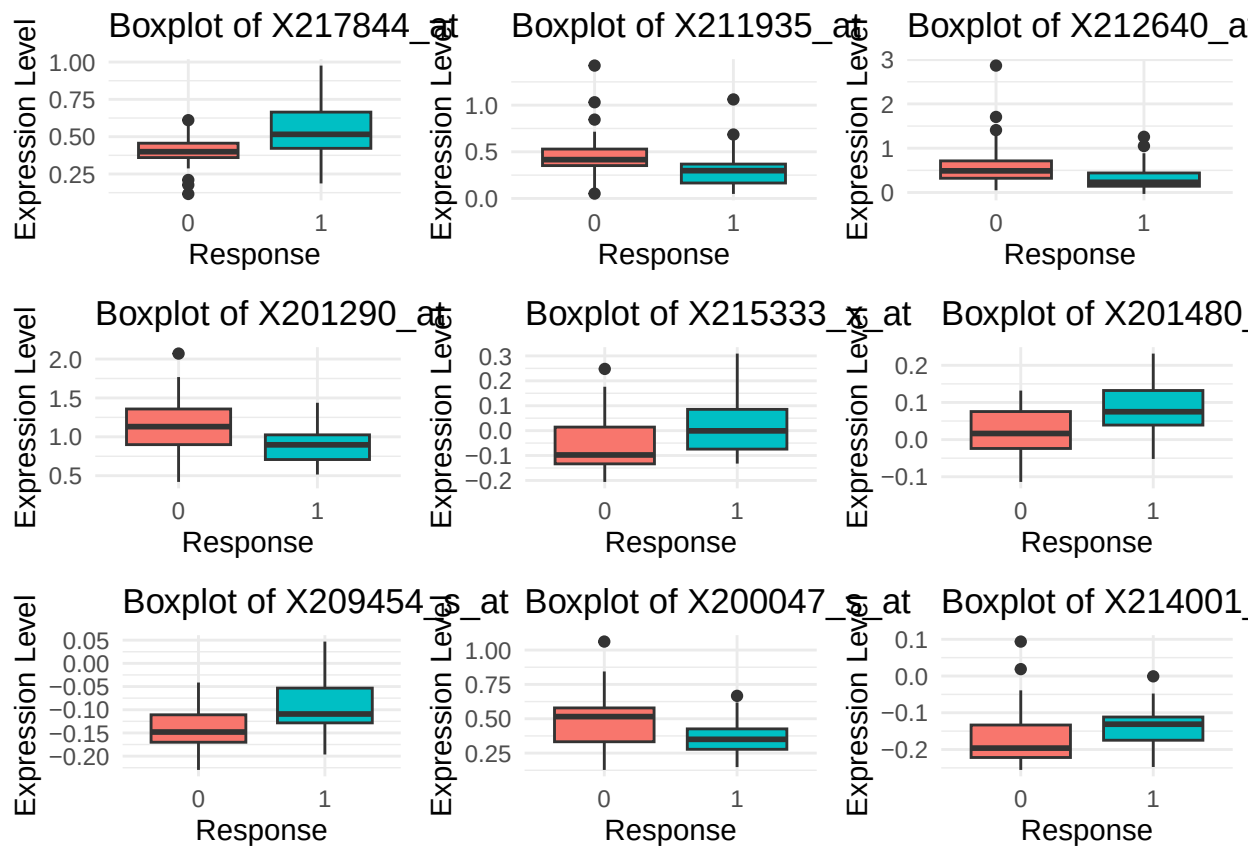
```
## [1] "X217844_at" "X211935_at" "X212640_at" "X201290_at" "X215333_x_at"
## [6] "X201480_s_at" "X209454_s_at" "X200047_s_at" "X214001_x_at"
```

5. Generate a type='h' plot with the Kruskal-Wallis test statistic as the y-axis and the variable name as the x-axis

Kruskal-Wallis Test Statistics (First 50 Variables)



6. Generate the comparative boxplots of the 9 most powerful variable with respect to the response and comment on what you observe.

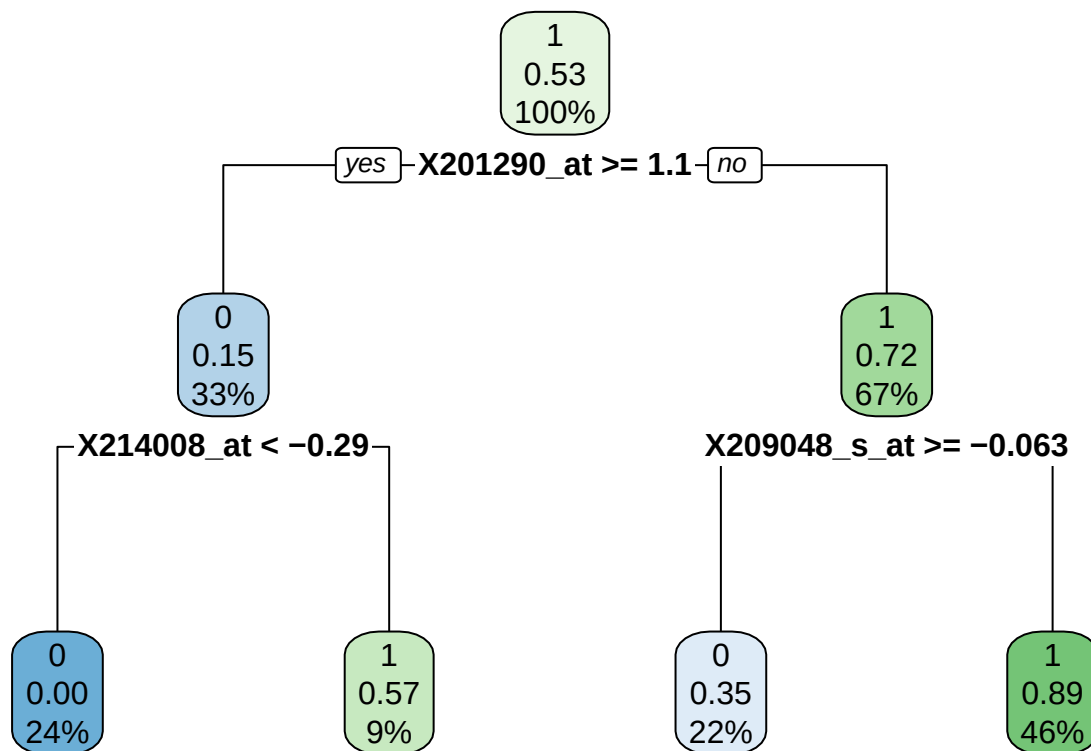


These variables show a clear distinction between the two response categories, suggesting that they can differentiate the response when taken alone.

7. Build the classification tree with $cp=0.01$

Plot the tree you just built

```
tree.xy <- rpart(Y~., data=prostate, control = rpart.control(cp = 0.01))
rpart.plot(tree.xy)
```



Determine the number of terminal nodes

```
length(tree.xy$frame$var[tree.xy$frame$var == "<leaf>"])
```

```
## [1] 4
```

```
print("The tree has 4 terminal nodes")
```

```
## [1] "The tree has 4 terminal nodes"
```

Write down in mathematical form region 2 and Region 4.

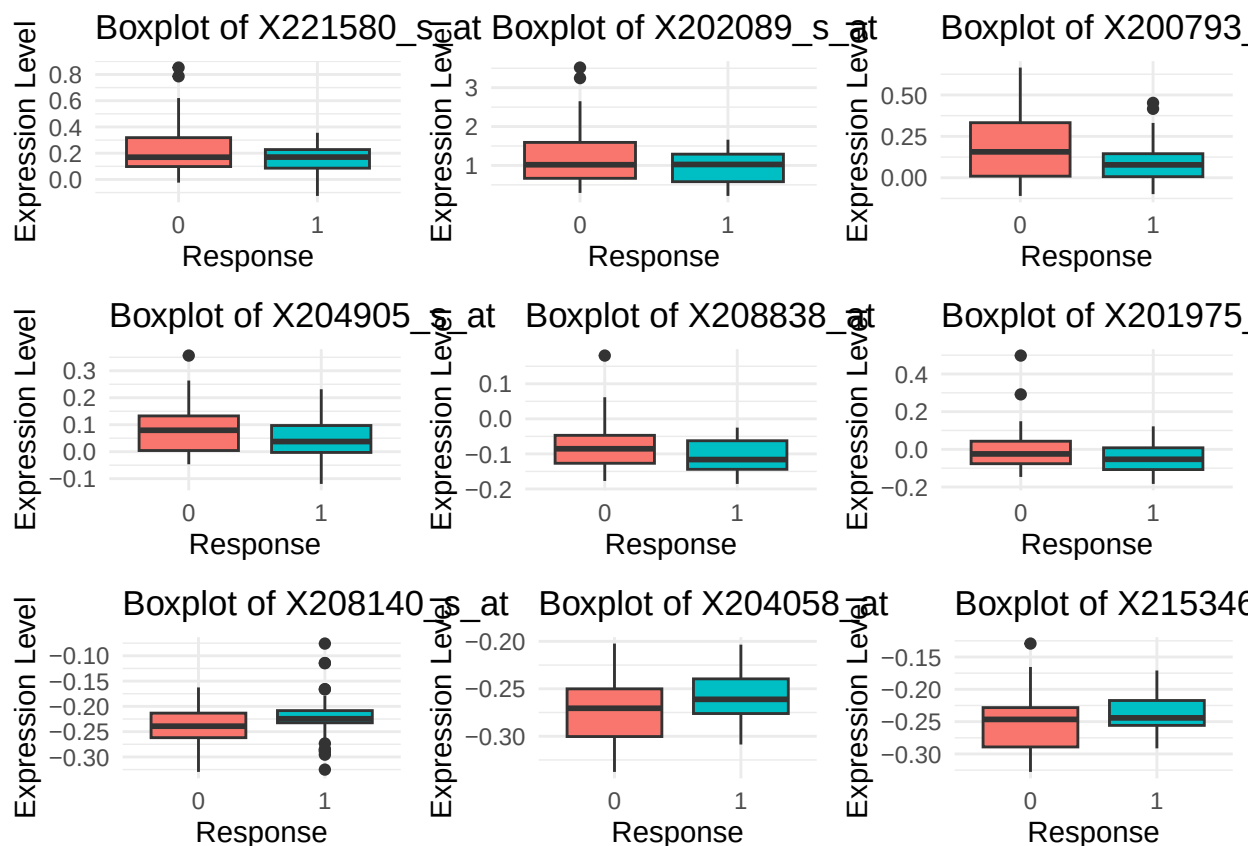
$$\text{Region 2} := \{X \in R^{500} | X_{201290} \geq 1.1 \text{ and } X_{209048} \geq 0.29\}$$

$$\text{Region 4} := \{X \in R^{500} | X_{201290} < 1.1 \text{ and } X_{209048} < -0.063\}$$

Comment on the variable at the root of the tree in light of the Kruskal-Wallis statistic

The variable at the root of the tree is X206788_s_at. In light of the Kruskal-Wallis statistic, we can see that "X206788_s_at" was the variable with the largest Kruskal-Wallis test statistic which means that it is the best differentiator between cancer and non-cancer subjects.

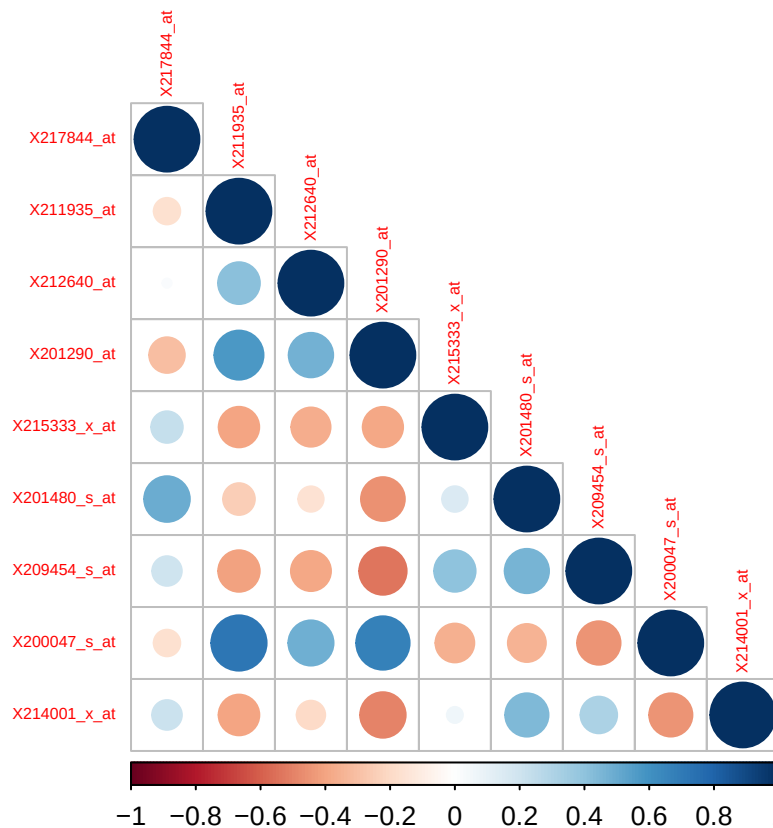
8. Generate the comparative boxplots of the 9 weakest variable with respect to the response and comment on what you observe.



The boxplots show that there is almost no separation between the expression levels of cancer and non-cancer subjects for these 9 variables, showing that these are weak features to use for classification.

9. Generate the correlation plot of the predictor variables and comment extensively on what they reveal, if anything.

```
top_9_prostate_data <- prostate[,c("Y",top_9_variables)]
cor_matrix <- cor(top_9_prostate_data[, 2:ncol(top_9_prostate_data)])
corrplot(cor_matrix, method="circle", type = "lower", tl.cex = 0.5)
```



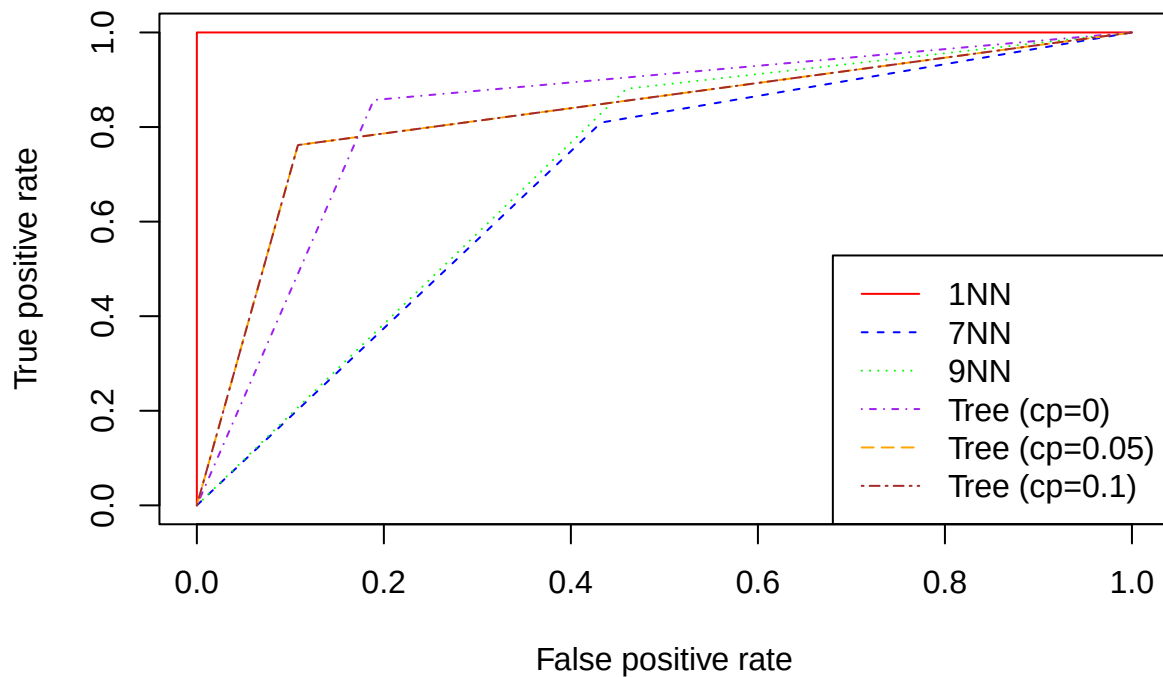
The correlation plot shows that the top 9 gene expression features are generally not highly correlated as most of the circles in the plot are small. However, there are a few clusters of variables which are moderately correlated which is observable in the dark shades present in the heatmap.

10. Compute the eigendecomposition of the correlation matrix and comment on the ratio

```
## Eigenvalues:
## 4.012561 1.345786 0.9879688 0.7034388 0.544242 0.4721013 0.3545035 0.3437462 0.2356522
## Ratio of max to min eigenvalues: 17.02747
```

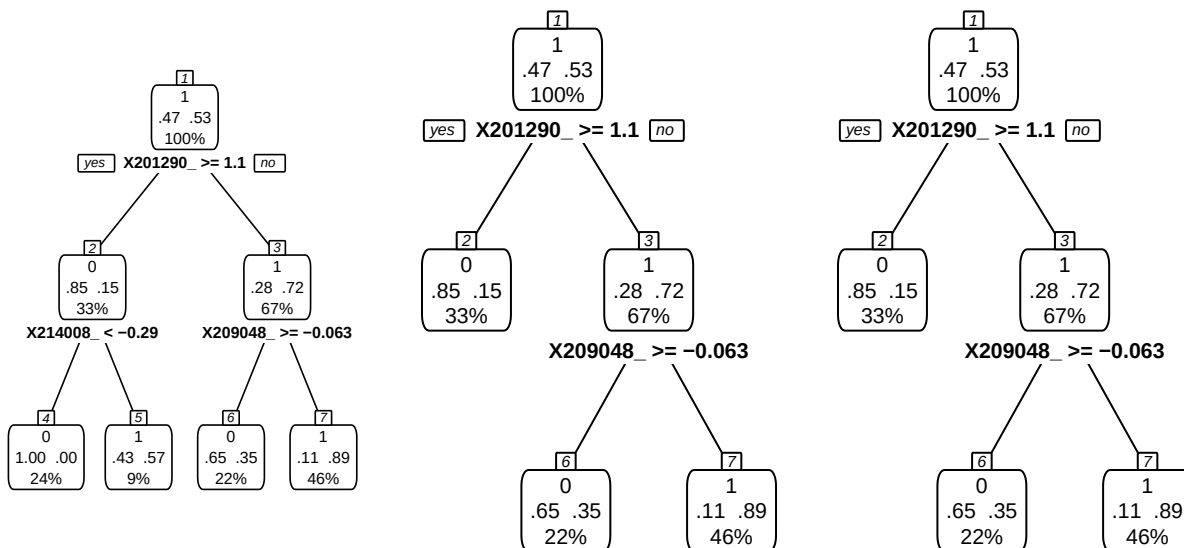
The ratio of the maximum to the minimum eigenvalues of the correlation matrix is quite high, which suggests some multicollinearity still exists in the selected feature space, this is because of the presence of some moderately correlated variable clusters shown in the correlation plot.

11. Using the whole data for training and the whole data for test, build the above six learning machines, then plot the comparative ROC curves on the same grid



12. Plot all the three classification tree grown, using the prp function for the package rpart.plot

```
par(mfrow = c(1, 3))
prp(tree_cp0, type=2, extra=104, nn=TRUE)
prp(tree_cp005, type=2, extra=104, nn=TRUE)
prp(tree_cp01, type=2, extra=104, nn=TRUE)
```

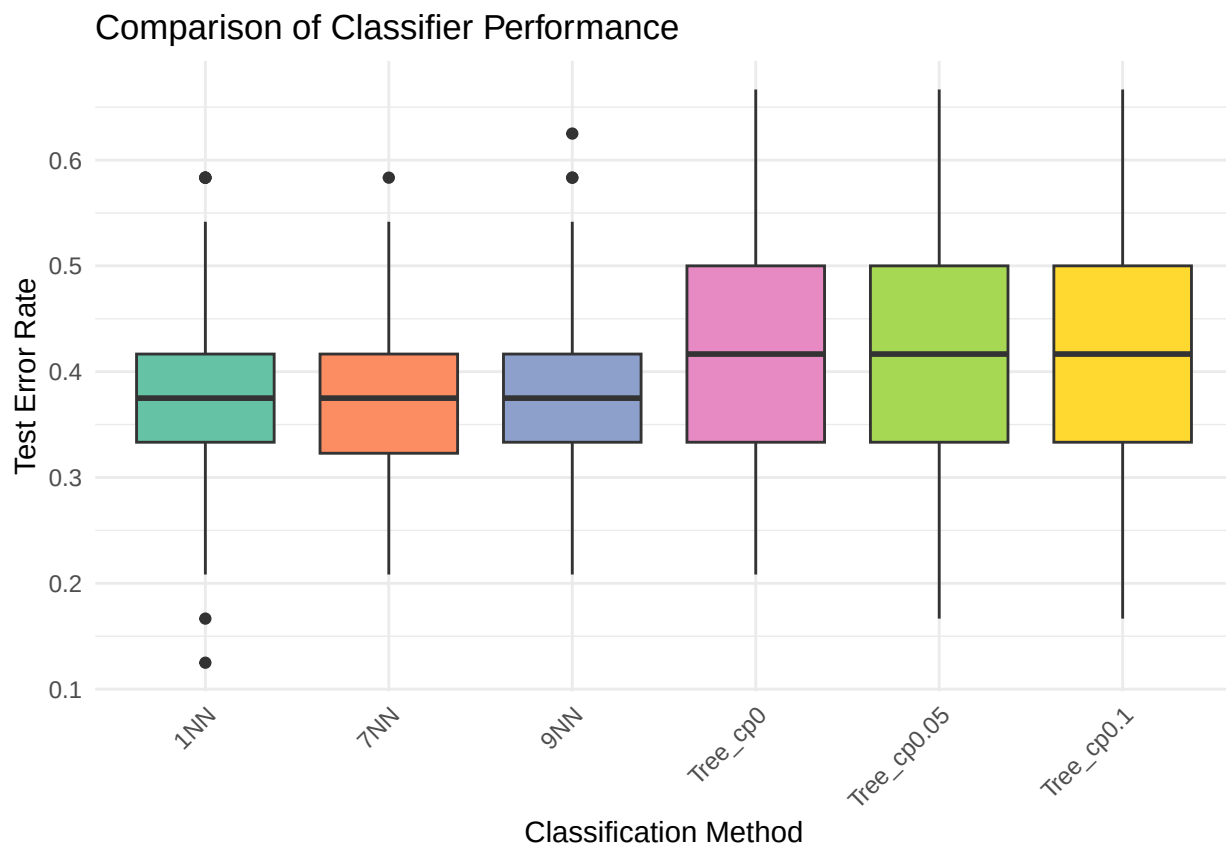


```
par(mfrow = c(1, 1))
```

13. Comment succinctly on what the ROC curves reveal for this data and argue in light of theory whether or not that was to be expected.

The ROC curves reveal that the 1NN model performs the best, followed by the 7NN and 9NN models. Among the classification trees, the model with $cp = 0$ performs the best, followed by $cp = 0.05$, and $cp = 0.1$ performs the worst. This aligns with the theoretical expectation that more complex models (like 1NN and trees with lower cp values) tend to fit the training data very well but may not generalize as well to unseen data. Simpler models, with higher cp value, are more robust to unseen data but may underfit the training data. The observed performance is consistent with the bias-variance tradeoff, where model complexity affects the balance between bias and variance.

14. Using `set.seed(19671210)` along with a 7/10 training 3/10 test basic stochastic holdout split of the data, compute $S = 100$ replicated random splits of the test error for all the above learning machines.



Comment on the distribution of the test error in light of (implicit) model complexity

k-NN Classifiers The test error rates are relatively stable across different values of k , with 1-NN showing slightly lower median test error rate but higher variability.

Decision Trees The test error rates decrease and become more stable as the complexity parameter (cp) increases, indicating that pruning improves generalization. The model `Tree_cp0` has the highest median test error rate and variability, suggesting overfitting.

Model Complexity More complex models (1-NN, Tree_cp0) tend to have higher variability in test error rates, while less complex models (9-NN, Tree_cp0.1) tend to have lower variability and potentially better generalization.

These observations highlight the trade-off between bias and variance in model complexity. Simpler models (higher k , higher cp) tend to have lower variance and potentially better generalization, while more complex models (lower k , lower cp) tend to have higher variance and potentially worse generalization.

Perform a basic analysis of variance (ANOVA) on those test errors and comment

```
anova_result <- aov(Test_Error ~ Method, data = test_errors_long)
summary(anova_result)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## Method         5  0.268  0.05363    6.739 4.02e-06 ***
## Residuals     594  4.728  0.00796
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ANOVA results indicate that there is a statistically significant difference in the test error rates among the different classification methods. Therefore, we can reject the null hypothesis that all classification methods have the same mean test error rate. The ANOVA analysis provides strong evidence that the classification methods have different impacts on the test error rates, and further analysis is warranted to identify the best-performing methods.

15. Comment extensively on the most general observation and lesson you gleaned from this exploration.

The primary lesson from this exploration is that we must use a test/training split to obtain a more valid evaluation of the learning machines. Using the whole dataset for both training and testing provides overly optimistic evaluation of model performances and does not reveal which models are more prone to overfitting. We see that when properly testing with the held out test dataset that the trees generally perform better and the more complex the tree, the lower the test error. However, increasing the cp parameter results in the simpler trees performing worse on the testing dataset than the more complex models, likely due to overfitting with higher model complexities. The ANOVA revealed that there is a statistically significant difference between the performances of the learning machines.

Exercise 2

Part 1: Multi-class classification on MNIST

```
mnist <- read_mnist()
xtrain <- mnist$train$images
ytrain <- mnist$train$labels
ytrain <- as.factor(ytrain)
ntr <- nrow(xtrain)
p <- ncol(xtrain)
xtest <- mnist$test$images
ytest <- mnist$test$labels
ytest <- as.factor(ytest)
```

1. Write down in mathematical form the expression of $\hat{f}_{\text{KNN}}(x)$ the prediction function of the kNN learning machine.

$$\hat{f}_{kNN}(x) = \arg \max_{g=1,\dots,G} \left(\frac{1}{K} \sum_{i=1}^k I(Y_i = g) \times I(X_i \in V_k(X)) \right)$$

2. Choose n a training set size and m a test set size, and write a piece of code for sampling a fragment from the large dataset. Explain why you choose the numbers you chose.

Explain why you choose the numbers you chose.

I decided to use 5% of the MNIST dataset it had a very large sample size of about 60000 Training and 10000 test. Using all the data would be computationally expensive. Also considering the computational power of my computer.

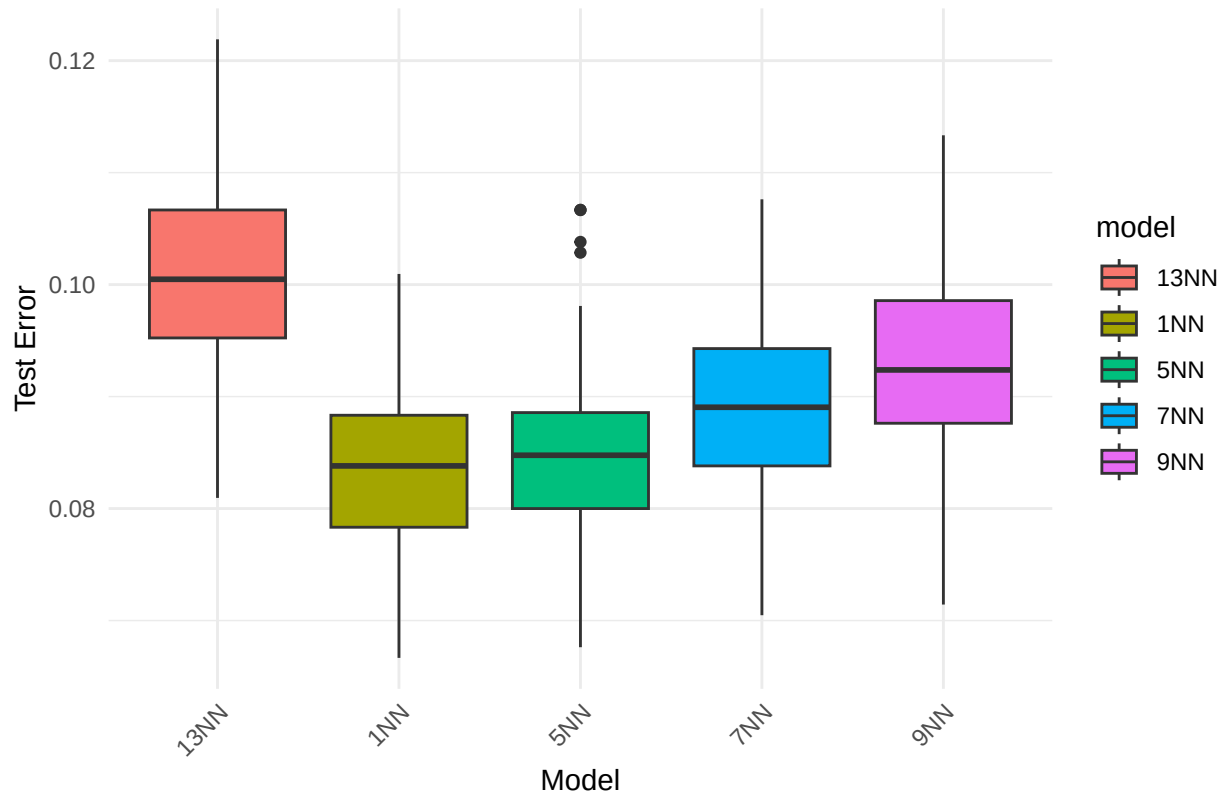
```
## Split 1: Training samples = 2450, Test samples = 1050
## Split 2: Training samples = 2450, Test samples = 1050
## Split 3: Training samples = 2450, Test samples = 1050
## Split 4: Training samples = 2450, Test samples = 1050
## Split 5: Training samples = 2450, Test samples = 1050
## Split 6: Training samples = 2450, Test samples = 1050
## Split 7: Training samples = 2450, Test samples = 1050
## Split 8: Training samples = 2450, Test samples = 1050
## Split 9: Training samples = 2450, Test samples = 1050
## Split 10: Training samples = 2450, Test samples = 1050
## Split 11: Training samples = 2450, Test samples = 1050
## Split 12: Training samples = 2450, Test samples = 1050
## Split 13: Training samples = 2450, Test samples = 1050
## Split 14: Training samples = 2450, Test samples = 1050
## Split 15: Training samples = 2450, Test samples = 1050
## Split 16: Training samples = 2450, Test samples = 1050
## Split 17: Training samples = 2450, Test samples = 1050
## Split 18: Training samples = 2450, Test samples = 1050
## Split 19: Training samples = 2450, Test samples = 1050
## Split 20: Training samples = 2450, Test samples = 1050
## Split 21: Training samples = 2450, Test samples = 1050
## Split 22: Training samples = 2450, Test samples = 1050
## Split 23: Training samples = 2450, Test samples = 1050
## Split 24: Training samples = 2450, Test samples = 1050
## Split 25: Training samples = 2450, Test samples = 1050
## Split 26: Training samples = 2450, Test samples = 1050
## Split 27: Training samples = 2450, Test samples = 1050
## Split 28: Training samples = 2450, Test samples = 1050
## Split 29: Training samples = 2450, Test samples = 1050
## Split 30: Training samples = 2450, Test samples = 1050
## Split 31: Training samples = 2450, Test samples = 1050
## Split 32: Training samples = 2450, Test samples = 1050
## Split 33: Training samples = 2450, Test samples = 1050
## Split 34: Training samples = 2450, Test samples = 1050
## Split 35: Training samples = 2450, Test samples = 1050
## Split 36: Training samples = 2450, Test samples = 1050
## Split 37: Training samples = 2450, Test samples = 1050
## Split 38: Training samples = 2450, Test samples = 1050
## Split 39: Training samples = 2450, Test samples = 1050
## Split 40: Training samples = 2450, Test samples = 1050
```

```

## Split 41: Training samples = 2450, Test samples = 1050
## Split 42: Training samples = 2450, Test samples = 1050
## Split 43: Training samples = 2450, Test samples = 1050
## Split 44: Training samples = 2450, Test samples = 1050
## Split 45: Training samples = 2450, Test samples = 1050
## Split 46: Training samples = 2450, Test samples = 1050
## Split 47: Training samples = 2450, Test samples = 1050
## Split 48: Training samples = 2450, Test samples = 1050
## Split 49: Training samples = 2450, Test samples = 1050
## Split 50: Training samples = 2450, Test samples = 1050

```

Comparative Test Error Boxplots



```
## Best model: 1NN
```

```

##           Reference
## Prediction  0  1  2  3  4  5  6  7  8  9
##           0 103  0  1  2  0  0  0  0  0  1
##           1  0 120  3  1  1  1  2  3  3  0
##           2  1  1 104  2  0  0  0  2  2  0
##           3  0  0  1 103  0  4  0  1  1  2
##           4  0  1  0  0  94  1  0  3  2  9
##           5  0  0  0  3  0  90  0  0  4  0
##           6  0  0  1  0  0  3  86  0  0  0
##           7  0  0  2  0  0  0  0  83  0  5
##           8  0  0  1  0  0  4  0  0  91  2
##           9  0  0  1  0  9  0  0  4  1  85

```

Comment on the digits for which there is a lot more confusion. Does that agree with your own prior intuition about digits?

In general, the confusion matrix supports common intuition about which digits are likely to be confused. Digits that have similar shapes or structures tend to be misclassified more frequently, such as 4 and 9, or 5 and 3. This is often the case in machine learning models for digit classification, especially when dealing with handwritten or stylized digits.

4. Perform an ANOVA of the test errors and comment on the patterns that emerge.

```
anova_result <- aov(test_error ~ model, data = boxplot_data)
summary(anova_result)
```

```
##              Df    Sum Sq   Mean Sq F value Pr(>F)
## model          4  0.008537  0.0021342   29.54 <2e-16 ***
## Residuals    245  0.017701  0.0000722
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The ANOVA results show that there is a statistically significant difference in the test errors across different KNN models. The F statistic is 26.3 with a p-value less than $2e-16$. This indicates that not all KNN models perform equally on this particular data.

Part 2: Binary classification on MNIST

1. Store in memory your training set and your test set. Of course you must show the command that extracts only '1' and '7' from both the training and the test sets.

```
## Training set dimensions: 3000 784
```

```
## Test set dimensions: 500 784
```

```
##
## K = 1
## Training Confusion Matrix:
```

```
##
##      1    7
## 1 1540    0
## 7    0 1460
```

```
## Test Confusion Matrix:
```

```
##
##      1    7
## 1 260    6
## 7    0 234
```

```
##
## K = 5
## Training Confusion Matrix:
```

```
##
##      1    7
## 1 1535   17
## 7    5 1443
```

```
## Test Confusion Matrix:
```

```
##
##      1    7
## 1 259    7
## 7    1 233
```

```
##
```

```

## K = 7
## Training Confusion Matrix:
##
##      1    7
##  1 1534   22
##  7    6 1438
## Test Confusion Matrix:
##
##      1    7
##  1 259    7
##  7    1 233
##
## K = 9
## Training Confusion Matrix:
##
##      1    7
##  1 1534   25
##  7    6 1435
## Test Confusion Matrix:
##
##      1    7
##  1 259    7
##  7    1 233
##
## K = 13
## Training Confusion Matrix:
##
##      1    7
##  1 1534   28
##  7    6 1432
## Test Confusion Matrix:
##
##      1    7
##  1 259    7
##  7    1 233

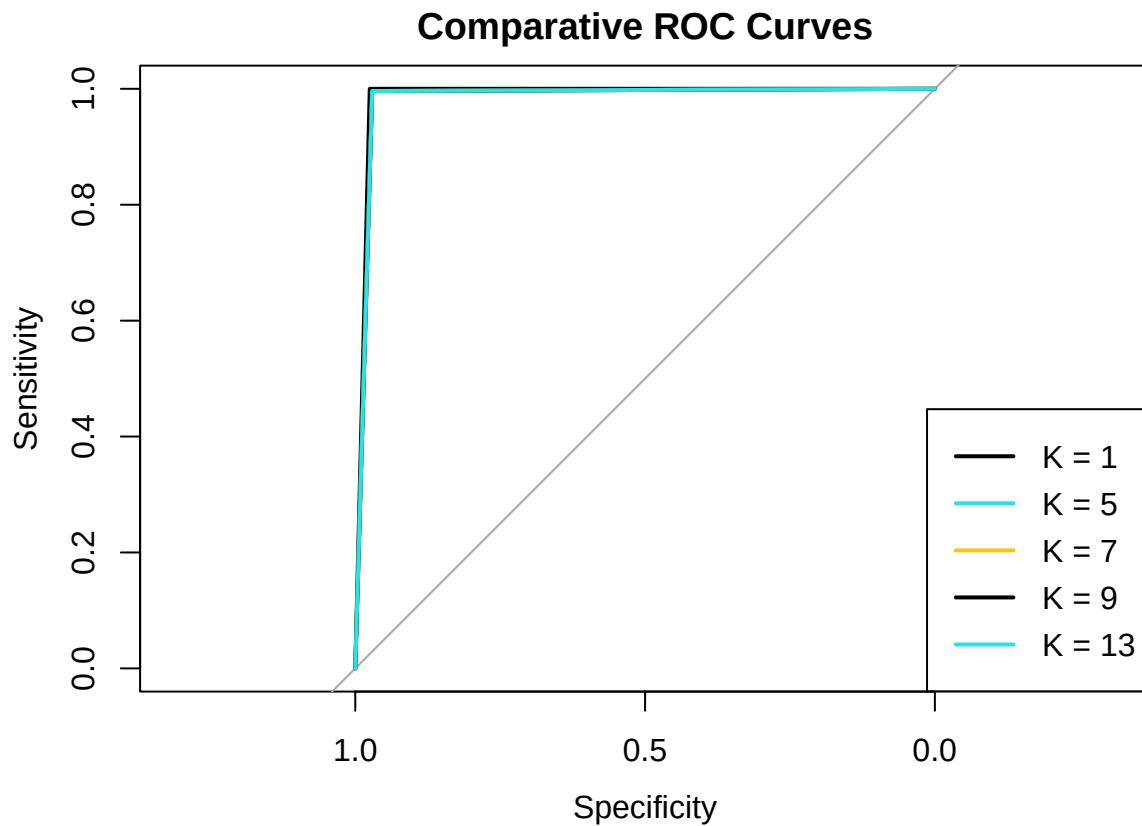
```

3. Display the comparative ROC curves of the five learning machines

```

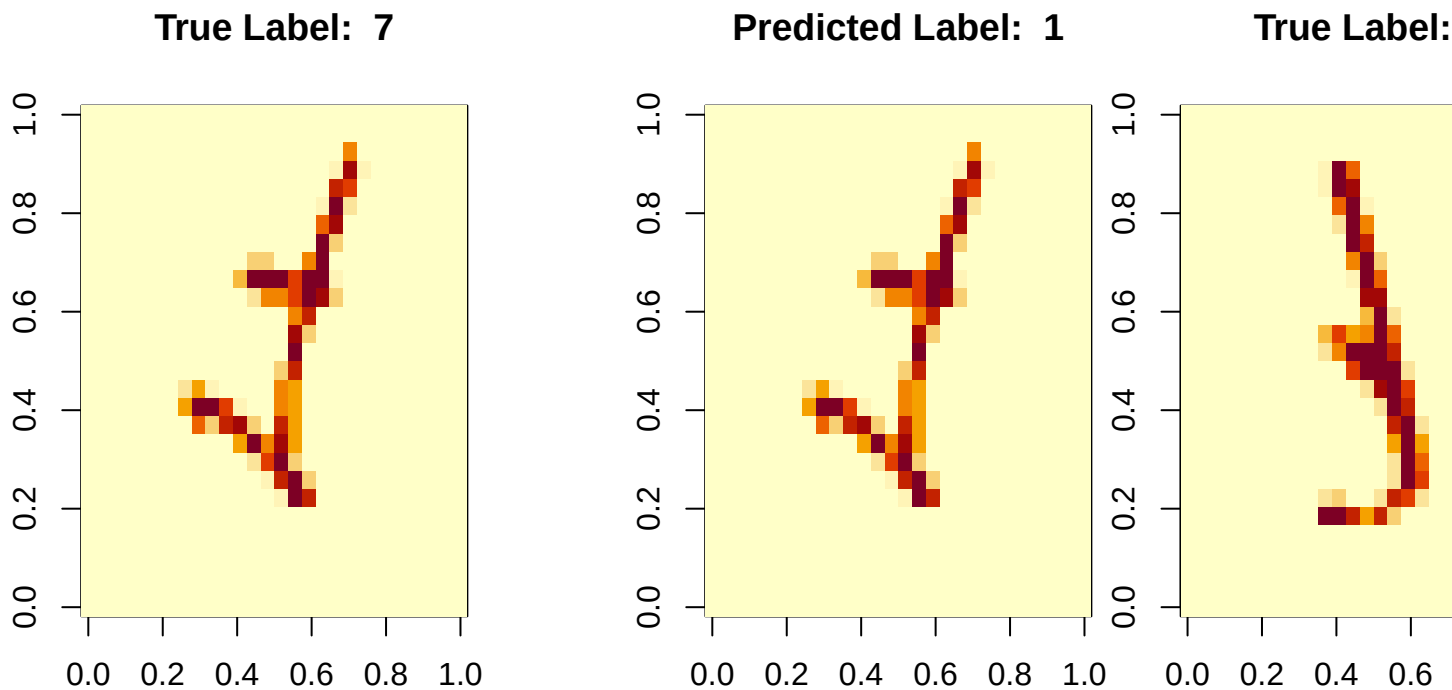
##
## K = 1
##
## K = 5
##
## K = 7
##
## K = 9
##
## K = 13

```



4. Identify two false positives and two false negatives at the test phase, and in each case, plot the true image against its falsely predicted counterpart.

Displaying False Positives:



Not enough false negatives for display.

5. Comment on any pattern that might have emerged.

I explored multi-class and binary classification using the MNIST dataset. For multi-class classification, I applied the k -NN algorithm with varying values of k . The 1NN model consistently performed best with the lowest median test error. The ANOVA results further confirmed significant differences in the performance of the models.

When focusing on binary classification for digits '1' and '7', I observed that the 1NN model again outperformed others, as evidenced by the ROC curves and confusion matrices. Upon inspecting false positives and negatives, I noticed a pattern where certain digits were frequently misclassified, indicating areas where the model struggled.

This insight highlights the importance of careful model evaluation and selection to improve classification accuracy.

Exercise 3
video link:

<https://youtu.be/VkF3N4Ate3c?si=gDFjiLkmogT22rTQ>